

Aufgabe 10

Aufgabe 10.1

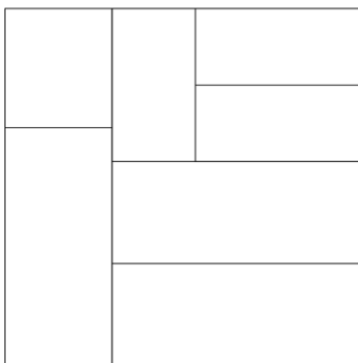
Erklärung:

Die Treemap zerlegt ein gegebenes Rechteck rekursiv in Teilrechtecke, deren Flächen proportional zu den Werten der Kindknoten sind – in dieser einfachsten Variante (*Slice-and-Dice*) wird dabei abwechselnd entlang der x- und der y-Achse geschnitten, beginnend mit der x-Achse auf der Wurzelebene. Damit das funktioniert, muss zuerst `sumUpInnerValues` post-order durch den Baum laufen und jedem inneren Knoten die Summe seiner Blattwerte zuweisen, denn die Originaldatei `flare.json` liefert nur an den Blättern ein `value`-Feld; ohne diesen Schritt wüsste ein innerer Knoten gar nicht, wie groß sein eigenes Rechteck im Verhältnis zu den Geschwistern sein soll. Das Zeichnen übernimmt `drawTreeNode` `splitX x y w h t`: es rahmt zunächst das aktuelle Rechteck als SVG-`rect` und teilt es dann entlang der aktiven Achse auf, wobei eine lineare Skala aus `elm-visualization` das Intervall `(0, totalValue)` auf die verfügbare Achsenlänge abbildet und `List.Extra.scanl (+) 0` die kumulierten Startpositionen der Kinder liefert; beim rekursiven Abstieg wird `splitX` negiert, sodass sich x- und y-Splits automatisch abwechseln. Das umgebende `drawTreemapSimple` setzt das Ganze in ein `500×500-SVG` mit `20 px` Padding, und zur Kontrolle wird der Baum zusätzlich als verschachtelte `<ul>`-Liste ausgegeben – so lassen sich die berechneten Werte direkt mit der grafischen Aufteilung vergleichen. Da der Halle-Server `flare.json` ohne CORS-Header ausliefert, läuft der Request in der Ellie-Variante über den Proxy `api.allorigins.win/raw`, während lokal die Datei einfach relativ zur HTML-Seite geladen wird.

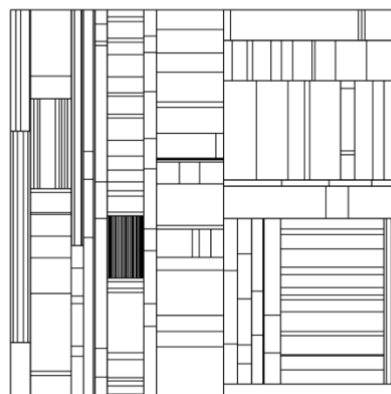
Screenshots:

Übung 10.1 – Einfache Treemap

testTree



flare.json



Baum als Liste

- flare 956129
 - analytics 48716
 - cluster 15207
 - AgglomerativeCluster 3938
 - CommunityStructure 3812
 - HierarchicalCluster 6714
 - MergeEdge 743
 - graph 26435
 - BetweennessCentrality 3534
 - LinkDistance 5731
 - MaxFlowMinCut 7840
 - ShortestPaths 5914
 - SpanningTree 3416
 - optimization 7074
 - AspectRatioBanker 7074
 - animate 100024
 - Easing 17010
 - FunctionSequence 5842
 - interpolate 23081
 - ArrayInterpolator 1983
 - ColorInterpolator 2047

Link: <https://ellie-app.com/yTsxzPxczsga1>

Aufgabe 10.2

Erklärung:

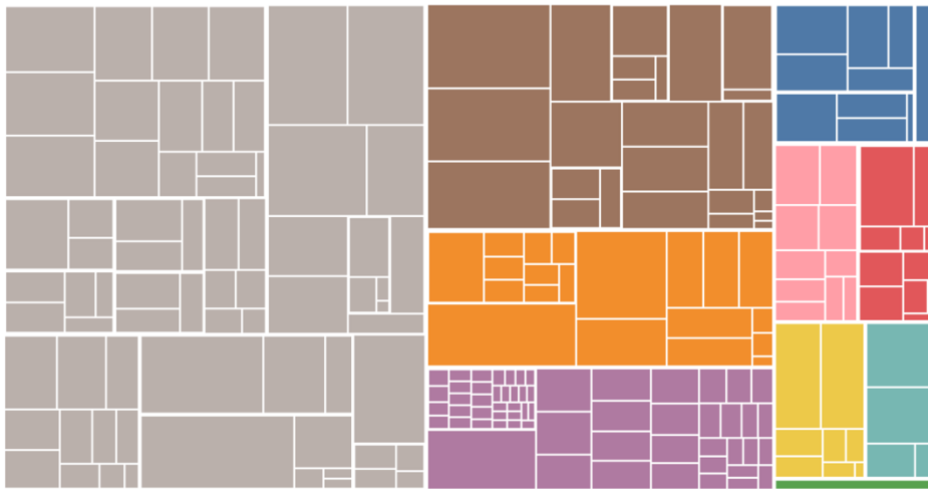
Aufgabe 10.2 nutzt statt der selbstgebauten Slice-and-Dice-Treemap aus 10.1 die fertige `Hierarchy.treemap`-Funktion aus `gampleman/elm-visualization` mit dem Tiling squarify – einem Algorithmus von Bruls et al., der bei jedem Split versucht, das Seitenverhältnis der entstehenden Rechtecke möglichst nahe an einem Quadrat (genauer: am goldenen Schnitt) zu halten und damit deutlich besser lesbare Treemaps als der einfache abwechselnde Slice-Ansatz liefert. Wie in 10.1 muss `flare.json` zuerst durch `sumUpInnerValues` laufen, damit jeder innere Knoten die Summe seiner Blattwerte trägt; anschließend wird der Baum nach Wert absteigend sortiert (das verbessert die Squarify-Heuristik) und mit den Optionen `tile squarify`, `padding 1` und `size 960 500` an `Hierarchy.treemap` übergeben, die für jedes Blatt ein Rechteck `{x, y, width, height, node}` zurückgibt. Die zusätzliche Anforderung – Färbung nach Teilbäumen ab Ebene 1 *und* ab Ebene 2 unter der Wurzel – wird gelöst, indem `annotate` einmal rekursiv durch den Baum läuft und jedem Knoten zwei zusätzliche Felder `group1` und `group2` mitgibt: den Namen seines Vorfahren auf Tiefe 1 bzw. Tiefe 2 (z. B. bekommen alle Knoten unter `flare.analytics.cluster` denselben `group2`-Wert "cluster"). Die Render-Funktion `drawTreemap` ist mit dem Gruppen-Accessor parametrisiert, sammelt die vorkommenden Gruppennamen in Erscheinungsreihenfolge und baut daraus eine `String → Color`-Tabelle aus mehreren zusammengesetzten kategorischen Paletten (`tableau10 ++ paired ++ set1 ++ set2 ++ category10 ++ accent`), damit auch

die deutlich zahlreicheren Ebene-2-Gruppen unterscheidbare Farben erhalten; bei mehr Gruppen als Farben wird zyklisch wiederverwendet. So entstehen zwei Treemaps mit identischer Geometrie, aber unterschiedlicher semantischer Gruppierung – einmal nach den großen Top-Level-Paketen, einmal nach den Unterpaketen eine Ebene tiefer.

Screenshots:

Übung 10.2 – Squarified Treemap

Färbung nach Teilbäumen ab Ebene 1 unter der Wurzel



Färbung nach Teilbäumen ab Ebene 2 unter der Wurzel



Link: <https://ellie-app.com/yTGsBnmVgFta1>